Assignment 4: IK, Trajectories, and Control

16-384: Robot Kinematics and Dynamics

Prof. Jeff Ichnowski
Albert Xu
Henry Liao
Karthik Srinivasan

Contents

1 Overview **3**

2 Background **4**
2.1 Analytical IK 4
2.2 Numerical IK 4
2.2.1 Advantages 5
2.2.2 Disadvantages 5
2.3 Trajectories 5
2.3.1 Bang-Bang Control (No Trajectories) 6
2.3.2 Constant Velocity (Better Than Nothing) 6

3 Instructions **8**

4 Code Questions **13**

5 Submission Checklist **25**

1 Overview

This assignment reinforces the following topics:

- Analytic IK
- Numerical IK
- Trajectories for Robot Control

2 Background

2.1 Analytical IK

One approach to IK is to solve for joint angles given a workspace end effector position manually, via algebraic methods or geometric analysis, for each different type of manipulator. This is very different from FK, which is pretty much formulaic, going through the transforms of each link to get the end effector position. The range of arms for which analytical solutions exist are only arms that are fully constrained, and solutions generally have to be further manually inspected to check for an arm exceeding joint limits or other practical considerations.

Analytical IK has the advantage that it is precise, fast (once you figure out the solution), and global, meaning that you can always find a solution for joint angles given an end effector position that is within the arm's workspace. So in short, with analytical IK:

- If there is a solution(s), you will find it.
- If there is no solution, you will know that.

A more flexible method we will discuss below is numerical IK, which can work around most of the limitations posed by things like redundancy and joint limits. However, it is approximate and local, meaning that you are not always guaranteed to find a 'good' solution.

2.2 Numerical IK

An alternative approach to analytical IK is numerical IK, or optimization-based IK. In this case we use the forward kinematics and a cost function to try to find a solution to the inverse kinematics. The general idea is to design a function that takes in a desired end effector pose and produces a scalar 'cost' (often an error based on distance to the desired pose) that an optimizer will try to minimize by varying the input variables. In this case the input variables are joint angles which are used to calculate the forward kinematics as part of the cost function.

Overall the process works like:

1. Pick a desired end effector pose
2. Pick an initial arm configuration (this is important detail; usually you are already in a configuration that you wish to start from)
3. Evaluate the forward kinematics at the current configuration
4. Evaluate the update rule for the input variables (e.g., gradient descent, a method based on the gradient of the cost function)
5. Check for 'stopping conditions', often based on the change in cost or the number of iterations (e.g., for gradient descent this can be related to the magnitude of the gradient); if we meet these stopping conditions, then return the current value of the input variables as our solution

6. Update the input variables and go back to step 3

Different optimizers have different methods of picking the next configuration to try, but most are based on gradient methods. Essentially, they try to figure out the slope of the cost function and trying to go 'downhill' until they reach a minimum.

There are distinct advantages and disadvantages of numerical IK compared to analytical IK.

2.2.1 Advantages

- Numerical IK is a 'black box'. As long as you have forward kinematics for an arm, you can make a cost function, pick an optimizer, and solve the inverse kinematics. It's general and relatively straightforward to implement.
- Numerical IK can handle arms that are under-constrained (redundant) or over-constrained.
- You can modify the cost function for numerical IK to include whatever you want. Joint limits, knowledge of obstacles, other high-level requirements, etc. can all be added relatively easily.

2.2.2 Disadvantages

- Numerical IK is a 'black box'. You do not necessarily get any intuition into the underlying structure of the problem. There might be multiple solutions to a desired pose, or none at all. You have to be careful with how you interpret and use the result that an optimizer spits out.
- Solutions are local, and subject to initial conditions. Optimizers all need an initial guess on where to start, and this guess can dramatically affect the solution the optimizer finds. The most reliable way to use numerical IK for arms is to start in a configuration that is as close as possible to the desired configuration, essentially 'warm-starting' the optimizer instead of searching over large areas of the configuration space.
- Numerical IK is usually less efficient computationally than analytical IK. Frequently this is not the limiting factor for controlling a robot. However, there are some cases where you might want to sample the workspace a huge number of times (one example of this in robotics research is the *RRT*, or Rapidly exploring Random Tree) and in such cases the speed of your IK might be the limiting factor.

2.3 Trajectories

When controlling a robotic arm, it is usually important to handle things in a way that is more sophisticated than just 'banging' the arm from one configuration to another. The general idea is to not command anything a robot physically do. Although we have not yet discussed dynamics in the class, dynamics affect the ability to control the position or velocity of any manipulator in the physical world. In short:

- Real robots have mass, m
- $F = ma$
- F is limited, so we should be careful about our desired a.

One way to mitigate effects of the dynamic limits of an arm is to use *trajectories* to smoothly move the arm from one configuration to another. (When thinking about trajectories, you can pose them as trajectories in work space or configurations space. Here we'll just think of trajectories in configuration space, but in the code problems you'll work with workspace trajectories as well.)

2.3.1 Bang-Bang Control (No Trajectories)

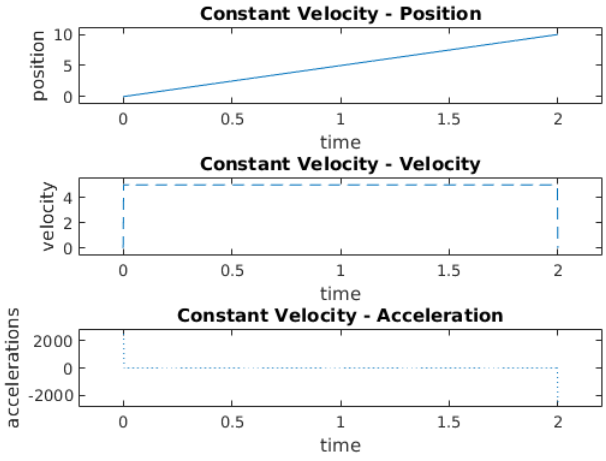
The simplest method is to ignore the pesky bounds of reality and rely on the underlying controller of the robot's actuators to 'catch up' to your commands. Depending on the specific application, and they way the low-level controllers are tuned, this can actually work. However, it has the disadvantage that the robot's motion is less predictable since the low-level controllers dictate the details of how the robot actually moves between the start and goal position – which is something that you often want to have control over! Additionally, with this approach you are usually 'jerking' the arm aggressively which can be dangerous and put unnecessary wear and tear on the arm's joints.

2.3.2 Constant Velocity (Better Than Nothing)

The next-simplest method you might think of would be to respect the reality that a robot's joints can't move with infinite speed. Looking at the case of moving a single DOF between two angles, we can choose an amount of time to take to move between the angles and command a corresponding constant velocity.

$T_{\text{move}} = (\theta_{\text{end}} - \theta_{\text{start}})/V_{\text{cruise}}$

This is a step in the right direction, but it has issues where we start and stop. To see the problem, let's think about the position, velocity, and acceleration.



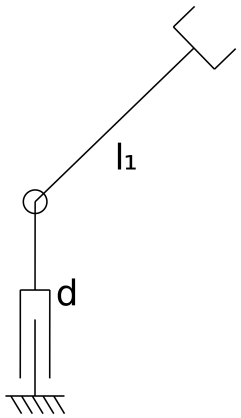
Because we jump from stop to a constant velocity and back again, at the ending steps we see big spikes in acceleration. In theory, these spikes are infinite acceleration, and in practice we will see some large finite value depending on how finely we discretize time (size of our Δt in the simulation or arm controller). Large accelerations are undesirable

because if the arm has any mass, then we are commanding large torques that are usually outside the capabilities of the robot. Again, the goal of using trajectories is to always command things the arm can physically do, so that it is playing 'catch up' as little as possible.

3 Instructions

- The deadline for this assignment is 1st October, 2026 11:00 A.M.
- The LaTEX format has been designed and tested on Overleaf, and **we strongly recommend using the Overleaf** to avoid any formatting errors as you fill out your answers.
- Zip your code into a single file named `<AndrewId>.zip`. See the complete submission checklist at the end, to ensure you have everything. Submit your PDF file to Gradescope.
- Each question (for points) is marked with a **points** heading.
- **Start early!** This homework may take a long time to complete.
- **During submission indicate the answer/page correspondence carefully when submitting on Gradescope.** If you skip a written question, just submit a blank page for it. This makes our work much easier to grade.
- If you have any questions or need clarifications, please post in Piazza or visit the TAs during the office hours.
- Unless otherwise specified, **all units are in radians, meters, and seconds**, where appropriate.

1) Analytical IK: PR robot
Given the PR Arm illustrated below from HW1.



a. [15 points] Analytically solve for d and θ, which result in an end-effector pose of $\begin{bmatrix} x_e \\ y_e \end{bmatrix}$.

$x = l_1 * c_1 \quad y = d + l_1 * s_1$
$d = y - l_1 * s_1 \quad \theta = \pm \arccos(\frac{x}{l_1})$
$d = y - l_1 * \pm \sin(\arccos(\frac{x}{l1})) = y \mp l1 * \sqrt{1 - (x/l1)^2}$

b. [5 points] Assume prismatic joint d can go from 0 and 1 and l_1 is 1. Using your

analytic IK from above, solve for d and θ that results in an end-effector position of $\left[\begin{smallmatrix}0.25\\1.5\end{smallmatrix}\right]$.

$\theta = \pm \arccos(\frac{0.25}{1}) \approx (1.318, -1.318)$
$d = 1.5 \mp \sqrt{1 - 0.25^2} \approx (0.532, 2.486)$
Solution: $(1.318, 0.532)$ (Other one violates joint limit)

c. [5 points] Assume prismatic joint d can go from 0 and 1 and l_1 is 1. Using your analytic IK from above, solve for d and θ that results in an end-effector position of $\left[\begin{smallmatrix}0.25\\0.5\end{smallmatrix}\right]$.

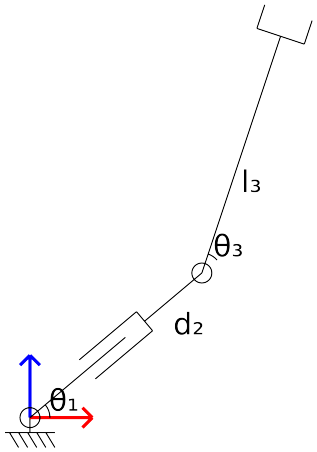
$\theta = \pm \arccos(0.25) \approx (1.318, 4.983)$
$d = 0.5 \mp \sqrt{1 - 0.25^2} \approx (-0.468, 1.468)$
There is no valid solution because both solutions violate the joint limits.

d. [5 points] Assume prismatic joint d can go from 0 and 1 and l_1 is 1. Using your analytic IK from above, solve for d and θ that results in an end-effector position of $\left[\begin{smallmatrix}-0.25\\-0.75\end{smallmatrix}\right]$.

$\theta = \pm \arccos(-0.25) \approx (1.824, -1.824)$
$f = -0.75 \mp \sqrt{1 - (-0.25)^2} \approx (-1.718, 0.218)$
Solution: $(-1.824, 0.218)$

2) Analytic IK: RPR robot

Consider the following robot



(Define the entire distance between the revolute joints as d_2).

a. [20 points] Geometrically solve for θ_1, d_2, and θ_3 which result in an end-effector pose of $\begin{bmatrix} x_e \\ y_e \\ \theta_e \end{bmatrix}$. Show your work! HINT: Break this problem down into an easier subproblem; are there known workspace positions of other parts of the robot, given the end effector pose?

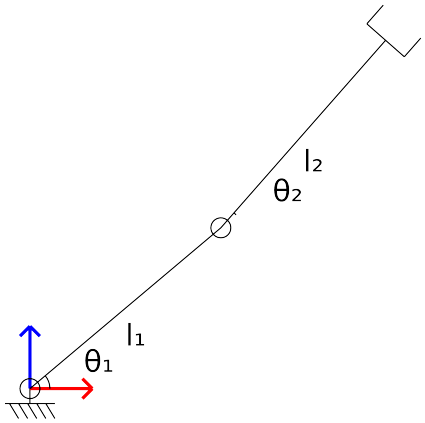
$$x_e = d_2 c_1 + l_3 c_{13} \quad y_e = d_2 s_1 + l_3 s_{13} \quad \theta_e = \theta_1 + \theta_3$$
$$x_1 = d_2 * c_1 \quad y_1 = d_2 * s_1 \rightarrow x_1^2 = d_2^2 c_1^2 \quad y_1^2 = d_2^2 s_1^2 \rightarrow x_1^2 + y_1^2 = d_2^2$$
$$x_1 = x_e - l_3 c_e \quad y_1 = y_e - l_3 s_e \rightarrow d_2 = \sqrt{(x_e - l_3 c_e)^2 + (y_e - l_3 s_e)^2}$$
$$\theta_1 = \text{atan2}(y_1, x_1) = \text{atan2}(y_e - l_3 s_e, x_e - l_3 c_e)$$
$$\theta_3 = \theta_e - \theta_1 = \theta_e - \text{atan2}(y_e - l_3 s_e, x_e - l_3 c_e)$$

b. [10 points] Assume there are no joint limits, except that $d_2 >= 0$. What (if any) limitations are there on achievable end-effector poses $\begin{bmatrix} x_e \\ y_e \\ \theta_e \end{bmatrix}$. Explain your answer in 1-2 sentences.

There are no limitations because there's no limitations on d2, so it can stretch as far as it wants, and then the two joints can hit any orientation, because θ_3 can fill the difference that θ_1 has.

3) Numerical IK: Cost function

Consider the following robot



a. [10 points] Let $p_e = \begin{bmatrix} f_x(\theta) \\ f_y(\theta) \end{bmatrix}$ (for forward kinematics f and joint configuration θ) be the end effector position as a function of the configuration θ. The goal is to find θ which achieves an end effector position as close as possible to the desired position $p_d = \begin{bmatrix} x_d \\ y_d \end{bmatrix}$. Let cost function h be the square of the distance between p_e and p_d. Write this cost function in terms of the forward kinematics function $f(\theta)$ and p_d (e.g. you may use f_x and f_y, but do not use l_1 or l_2).

$$h(\theta) = \frac{1}{2}\left(\begin{bmatrix} f_x(\theta) \\ f_y(\theta) \end{bmatrix} - p_d\right)^T \left(\begin{bmatrix} f_x(\theta) \\ f_y(\theta) \end{bmatrix} - p_d\right)$$

b. [10 points] Write the gradient of h, $\nabla h(\theta) = [\frac{\partial h}{\theta_1}, \frac{\partial h}{\theta_2}]^T$, in terms of f, p, and partial derivatives of f with respect to θ_i. Note: do not actually write out the derivatives of f; just use, for example, $\frac{\partial f}{\theta_1}$. HINT: if you are not familiar with vector differentiation, first write out $h(\theta)$ as a scalar expression by multiplying the individual vectors, and then compute the required partial derivatives as given above in the definition of $\nabla h(\theta)$.

$$h(\theta) = \tfrac{1}{2}\left(\begin{bmatrix} f_x(\theta) - x_d & f_y(\theta) - y_d \end{bmatrix}\right)\left(\begin{bmatrix} f_x(\theta) - x_d \\ f_y(\theta) - y_d \end{bmatrix}\right) =$$
$$\tfrac{1}{2}\left((f_x(\theta) - x_d)^2 + (f_y(\theta) - y_d)^2\right)$$

$$\frac{\partial h}{\partial \theta_1} = (f_x(\theta) - x_d)\frac{\partial f_x}{\partial \theta_1} + (f_y(\theta) - y_d)\frac{\partial f_y}{\partial \theta_1} \quad \frac{\partial h}{\partial \theta_2} = (f_x(\theta) - x_d)\frac{\partial f_x}{\partial \theta_2} + (f_y(\theta) - y_d)\frac{\partial f_y}{\partial \theta_2}$$
$$\nabla h(\theta) = \begin{bmatrix} (f_x(\theta) - x_d)\frac{\partial f_x}{\partial \theta_1} + (f_y(\theta) - y_d)\frac{\partial f_y}{\partial \theta_1} \\ (f_x(\theta) - x_d)\frac{\partial f_x}{\partial \theta_2} + (f_y(\theta) - y_d)\frac{\partial f_y}{\partial \theta_2} \end{bmatrix}$$

c. [10 points] Reorder these terms to write this expression in terms of the end effector Jacobian, J, along with f and p.

$\nabla h(\theta) = J(\theta)^T(f(\theta) - p_d)$

d. [10 points] What would the expression for the gradient from the previous part look like for a 100 link serial chain of revolute modules? Is there any difference? Describe why or why not.

There wouldn't be any difference because it's the same pattern, but it would be expanded by 100 points. These would be accounted for in J with it expanding to an 2×100 and θ being a 1×100. f and p_d would stay the same because they're still x and y

4 Code Questions

In these problems you will extend the `Robot` class from the last assignment with Jacobians and inverse kinematics, use them to build trajectories, and compute the torques that hold a real robot arm up against gravity. Each part can also be run on a simulated xArm7, the arm used in the lab.

Setup

Copy the code handout to a location of your choice. Everything below is run from the top level of that folder, the one containing `local_autograder.py`.

If you have not installed it yet, or you end up getting package errors later on down the line, conside reinstalling the course's robot library. The handout includes its own copy of the library, in `16384-robot-lib`. It carries the simulator, the visualizer and the collision checker the demos use, and it may be newer than the version you installed for an earlier assignment. Update your environment from it:

```
cd 16384-robot-lib
conda env update -f environment.yml    # new machine: conda env create
conda activate 16384
cd ..
```

This installs the library as the Python package `xarm7_lib`, pointing at the copy in your handout.

The four exercises live in their own directories:

- `ex1` — forward kinematics, the numerical Jacobian, and two scripts that use it.
- `ex2` — numerical inverse kinematics.
- `ex3` — joint-space and workspace trajectories.
- `ex4` — gravity compensation for the xArm7.

`ex1`, `ex2` and `ex3` each contain a copy of `robot.py`. Each exercise is graded with the copy in its own folder, so once a method works in one, copy it into the others: `ex2` needs your forward kinematics and Jacobians, and `ex3` needs both the work from `ex1` and your inverse kinematics from `ex2`.

Testing your work

`local_autograder.py` checks your functions against a set of stored test cases. Run all of them, or one exercise at a time:

```
python local_autograder.py               # everything
python local_autograder.py -p ex1        # just ex1 (or ex2, ex3, ex4)
python local_autograder.py --verbose     # list every failing case
```

A failing run lists the first few cases that did not match, and why. The stored cases live in `exN/inK.txt` (inputs) and `exN/outK.txt` (expected outputs).

A word of warning. The autograder on Gradescope does not use those stored outputs. It takes the same inputs, perturbs them by a small random amount that is different on every submission, and compares your result against a reference implementation. Code that reproduces the published numbers without actually computing the answer will score zero there. Write the general solution and the two graders will agree.

There is one deliberate difference between them. `sample_velocities.py`, `jacobian_example.py` and `linear_workspace_trajectory.py` all call a `Robot`. Locally they are given your own `robot.py`, so they can only pass once your `Robot` does. On Gradescope they are given a correct `Robot`, so there they are graded on their own.

Seeing your code on the xArm7

Every exercise has a demo that runs your code on a simulated UFACTORY xArm7:

```
python local_autograder.py -p vis1    # ex1; vis2 to vis4 for the rest
```

Each one opens a browser tab with sliders above a 3D view of the arm. Press ctrl-c in the terminal to stop it. The demos work before you have written anything: the arm still moves, and the terminal says which of your functions isn't ready yet and why.

The xArm7 has seven revolute joints, and `ex1`–`ex3` are about planar arms. The two are the same mechanism once four of the seven joints are held still. The library's lab pose holds

$$\theta_2 = \frac{\pi}{2}, \quad \theta_3 = \frac{\pi}{2}, \quad \theta_5 = -\frac{\pi}{2}, \quad \theta_6 = \frac{\pi}{2},$$

which leaves joints 1, 4 and 7 with their rotation axes all parallel to the world z-axis. What can still move is then exactly a planar RRR arm: joint 1 at the shoulder, joint 4 at the elbow, joint 7 at the wrist, with a drawn 10 cm tool as the third link. The demos measure the link lengths off the robot model and build your `Robot` with them, so your code is run on the real geometry of the arm.

1) [40 points] Forward Kinematics and the Numerical Jacobian

Complete `ex1/robot.py`, `ex1/sample_velocities.py` and `ex1/jacobian_example.py`.

a. [15 points] **Forward kinematics**

Reimplement or copy your `forward_kinematics` from the last assignment into the marked section of `ex1/robot.py`. It returns a $3 \times 3 \times (n+1)$ array of homogeneous transforms. `frames[:, :, i]` is the frame at the base of link i, already rotated by joint i, and `frames[:, :, -1]` is the end effector, a translation along the last link with no further rotation.

b. [15 points] **Numerical Jacobian: implementation**

The next task is to write the last Jacobian you'll ever need: one that works for any frame generated by the forward kinematics. Fill in the `jacobians` method in `ex1/robot.py`.

This function computes a $3 \times n$ Jacobian (where n is the number of joints) for each of the $n+1$ frames, and returns them stacked as a $3 \times n \times (n+1)$ array. The code describes the conventions used for the arguments and result.

Whereas in previous assignments you have calculated the analytic derivative for each element of the Jacobian, here you will use a numerical approach. Given a general function $g(x)$, one can approximate its derivative $g'(x)$ by the forward difference $(g(x+\epsilon) - g(x))/\epsilon$, or more accurately by the central difference $(g(x+\epsilon) - g(x-\epsilon))/(2\epsilon)$.

In the code, we will take the same approach for each element of the Jacobian. For example, for the element corresponding to the x value for frame i and joint j (given a joint configuration $\mathbf{Q}$), evaluate the forward kinematics f_x at $\mathbf{Q} + \Delta\mathbf{q}_j$ (and, for a central difference, at $\mathbf{Q} - \Delta\mathbf{q}_j$), where $\Delta\mathbf{q}_j$ is a vector of zeros with ϵ for element j, and divide the difference by ϵ (or 2ϵ). Use the $\epsilon = 0.001$ given in the code; either difference is accepted. As with the analytical approach, the rotation row is filled in by inspection for now.

c. [5 points] **Verification: sample velocities**

Fill in `compute_kinematics` in `ex1/sample_velocities.py`. For every time step it computes the end effector position with the forward kinematics, and its velocity by multiplying the end effector Jacobian by the joint velocities. Then run the script:

```
cd ex1
python sample_velocities.py
cd ..
```

Verify that:

(i) On the first plot, the paths match *and* the end effector velocity vectors are tangent to the path that it takes.

(ii) On the second plot, the x and y velocities match.

Also, use your `jacobians` to find the Jacobian for an RRR arm at configuration $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$. Look at the third row of the Jacobian for each frame, the effect of joint angle velocity on the orientation of the frame. For the first frame (recall, attached to the first link), only the first joint angle should have an effect (i.e., this row should be $[1, 0, 0]$). For the second frame, the first and second joint angles should have an effect. The third and fourth frames (third link and end effector, respectively) should each be affected by the motion of every joint.

d. [5 points] **Application: joint torques for a wrench**

Because we have a Jacobian for any planar revolute-joint based arm, we can now address problems that would have been much more difficult before. Imagine a RRRRRRRRRR arm (10 R's), with links each equal to 5. Using the `Robot` class you have written, compute the joint torques necessary to apply a wrench[1] of $\begin{bmatrix} 0 \\ 5 \\ 2 \end{bmatrix}$ with

[1] Recall this is a combined force and torque vector; the first and second terms are forces in x and y, and the third term is a moment.

the end effector when the robot is at the following configuration:

$$\boldsymbol{\Theta} = \begin{bmatrix} 0, 0.1, 0.2, 0.3, 0.4, 0, -0.1, -0.2, -0.3, -0.4 \end{bmatrix}^T$$

Fill in `compute_joint_torques` in `ex1/jacobian_example.py`: it takes a robot, its joint angles and a wrench, and returns $(\mathbf{J})^\top \mathbf{w}$ for the end effector Jacobian. Running the script (`cd ex1; python jacobian_example.py`) builds the 10R arm above and prints the torques.

(Just for fun, imagine the number of terms for the analytic Jacobian for this arm. If you could write that out the very first time without making a mistake or a sign error, you might be the first person to ever do so…)

e. **See it on the arm**

```
python local_autograder.py -p vis1
```

The page has two rows of sliders. The `theta` sliders set the three joint angles and the arm follows them. The `dtheta` sliders set a joint velocity $\dot{\boldsymbol{\Theta}}$; the arm does not move with these, they only set the velocity the arrows below are drawn for.

Over the arm, the demo draws:

- a yellow skeleton through the frame origins your `forward_kinematics` returns, with an axis triad at your end effector frame. When your kinematics are right the skeleton lies inside the arm and the triad sits on the red tool tip.
- a **blue** arrow: your end effector velocity, $\mathbf{J}\,\dot{\boldsymbol{\Theta}}$, from your `jacobians`.
- a **green** arrow: the tip velocity the arm actually has at that joint velocity, computed from the arm itself. When your Jacobian is right the blue arrow covers the green one.

The status line under the sliders gives the numbers: your tip position against the measured tip, and your velocity against the arm's. Try setting one `dtheta` slider at a time and check that each arrow is perpendicular to the line from that joint to the tip.

2) [25 points] Numerical Inverse Kinematics (IK)
Complete `ex2/robot.py`.

a. **Implementation**

The next extension to the `Robot` class is a function to compute inverse kinematics using gradient descent, as discussed in lecture. Fill in the `inverse_kinematics` method in `ex2/robot.py` (after copying in your forward kinematics and Jacobians from `ex1`).

This function accepts a vector of joint angles for initial conditions and a desired end effector location. It returns the resulting joint angles that are the solution of the gradient descent process. The code describes the conventions used for the arguments and result, and gives the step size, stopping condition and iteration limit to use.

For this problem, assume the cost function is the sum of squared errors between the end effector and the goal point, ignoring joint angle limits. Use the solution to the second written problem to help compute the gradient of the cost function, and use a loop to continue updating the joint angles until no significant progress is made. **REMINDER**: You use the *negative* gradient in this process, multiplied by a small step size α.

Your code should handle two cases:

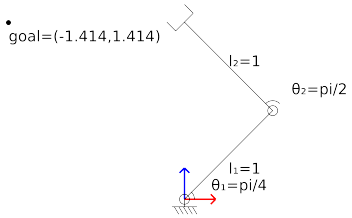
- If the goal is a 2×1 vector, it should only consider the $[x, y]^T$ position of the end effector.
- If the goal is a 3×1 vector, it should consider the $[x, y, \theta]^T$ pose of the end effector.

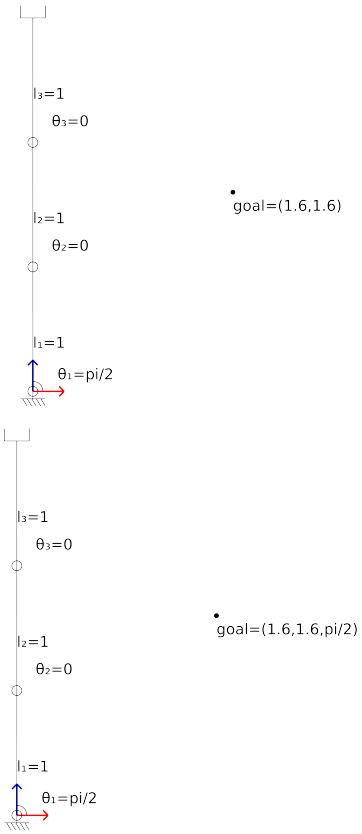
HINT 1: Note that the `end_effector` method in the `Robot` class returns the end effector's $[x, y, \theta]^T$ pose. This will be very useful in coding the equation!

HINT 2: For the 3×1 goal vector case, note that the form of the solution does not change; you must simply use an additional component of the pose and update your calculations accordingly.

b. **Verification**

To verify the results, solve the examples illustrated in the following figures:





Note: You do not have to submit anything for this part; however, you should use this to detect any problems with your code before continuing!

All three are set up in `ex2/IK_example.py`, which prints the final joint angles and the end effector pose they reach:

```
cd ex2
python IK_example.py
cd ..
```

For example, the first problem is:

```
robot = Robot(
    link_lengths=np.array([1, 1]),
    link_masses=np.array([1, 1]),
    joint_masses=np.array([1, 1]),
    end_effector_mass=1
)
initial_angles = np.array([np.pi / 4, np.pi / 2])
goal = np.array([-np.sqrt(2), np.sqrt(2)])
final_angles = robot.inverse_kinematics(initial_angles, goal)
```

For these three problems, the resulting joint angles should move the end effector toward the goal when passed into the forward kinematics, which you can check with `robot.end_effector(final_angles)`. With the given step size and 100 iterations, the first example gets close but not all the way there; that is expected.

Note that for examples 2 and 3, the resulting joint angles are different because of the inclusion of the end effector orientation constraint.

A caution here is that numerical methods for IK are *local*. For example, the first problem above has two solutions (elbow up/elbow down), but numerical IK only finds the one closest to the initial configuration (or in the case of constraints such as joint angles, may not find any valid solution). This is important to consider for real-world applications.

This local approach can be a benefit over analytical methods, though: for example, in the first problem, if the goal was $[-2, 2]^T$, analytical IK would return no solution, whereas numerical methods would still minimize the cost as best as possible and result in a "close" solution.

Another note: you can make the optimization more accurate by decreasing the step size and stopping condition; however, for our 'plain' gradient descent implementation, this can result in long runtimes. There are built-in numerical optimization methods in Python (e.g., from `scipy.optimize`) that are much more efficient than our simple approach here, resulting in more accurate results in less time.

c. **See it on the arm**

```
python local_autograder.py -p vis2
```

Drag the `goal x` and `goal y` sliders to place a goal, drawn as a purple ball. Once the goal has stopped moving for a moment, your `inverse_kinematics` runs from the arm's current joint angles and the arm is sent to the answer. Set `match angle` to 1 to use an $[x, y, \theta]^T$ goal instead; the `goal angle` slider then sets θ, drawn as a short arrow from the goal.

The status line reports how far the arm's measured tip ended up from the goal, in millimetres, and how many times your IK was called. Your descent's step size was chosen for arms with metre-long links, like those in the examples above; the xArm's links are about a third of that, the gradient is correspondingly smaller, and one call only gets part of the way. So the demo calls your IK again from its own answer until it stops moving. If your answer would make the arm collide with itself, the status line says so and the arm stops short. Try goals on both sides of the arm and watch which elbow configuration your IK picks.

3) [20 points] Trajectories

When controlling a robot, one option is to just command a destination joint angle configuration and allow the robot's internal joint angle controllers to move the joints towards this goal. However, depending on the starting position of the robot, this can result in large dangerous motions and will not give you any control over *how* the robot gets to this destination. Are there obstacles you want the robot to avoid? Do you want the resulting motion to be smooth?

Instead, one can define a *joint angle trajectory* for the robot's joints to follow through this motion to the goal point and then iteratively command points along this trajectory to result in a smooth, controlled motion along a well-defined path.

a. [5 points] **Straight Line Joint Trajectories**

Given a starting joint configuration and a goal joint configuration, one option is to connect the start and ending joint angles with a straight line in *joint space*. This means that you linearly interpolate between the start and end joint angles[2].

Fill in the `linear_joint_trajectory` function in `ex3/linear_joint_trajectory.py`. This function takes a NumPy array of start angles, a NumPy array of end angles, and a trajectory resolution (number of points). It linearly interpolates between the angles to reach the end points at the given number of points. It returns a matrix of n angles by t timesteps.

b. **Verification**

For an RRR arm with unit link lengths, interpolate between the start state

$$\boldsymbol{\Theta}_{\text{start}} = \begin{bmatrix} \pi/4 \\ \pi/2 \\ \pi/2 \end{bmatrix}$$

and the end state

$$\boldsymbol{\Theta}_{\text{end}} = \begin{bmatrix} 0 \\ \pi/2 \\ 0 \end{bmatrix}$$

with 100 steps. `ex3/verify_linear_joint_trajectory.py` does exactly this and passes the result to `visualize_trajectory`:

```
cd ex3
python verify_linear_joint_trajectory.py
cd ..
```

The joint angle plots should be straight lines, as this is a linear interpolation in joint space. What does the end effector path look like? Does this make sense?

Note: You do not have to submit anything for this part; however, you should use this to detect any problems with your code before continuing!

c. [15 points] **Linear Workspace Joint Trajectories**

Now assume that you have a start joint configuration and want to reach a workspace goal position g by moving in a straight line in *the workspace*. This can

[2] In Python, the function `numpy.linspace` will linearly interpolate between two numbers with a given number of points.

be useful for moving between obstacles or approaching obstacles for manipulation.

Fill in the `linear_workspace_trajectory` function in `ex3/linear_workspace_trajectory.py`, and copy your inverse kinematics into `ex3/robot.py`. This function takes a `Robot` object, a NumPy array of start angles, a workspace goal position (it should accept points in both $\mathbb{R}^2$ and SE(2)), and a trajectory resolution (number of points). It should return a matrix of n angles by t timesteps which result in a straight line workspace trajectory between the inputs.

The general approach to this problem is to first linearly interpolate between the initial and goal workspace positions and then solve the inverse kinematics (IK) for each of these positions in turn. Note that when solving for the ith column of the resulting trajectory matrix, you should use the $(i-1)^{\text{th}}$ column as the initial condition for the numerical IK, as this will ensure the resulting trajectory is smooth.

d. **Verification**

We will repeat the verification from the last part of this problem, except we use the workspace position for the end point instead of the joint angles. For an RRR arm with unit link lengths, interpolate between the start state

$$\boldsymbol{\Theta}_{\text{start}} = \begin{bmatrix} \pi/4 \\ \pi/2 \\ \pi/2 \end{bmatrix}$$

and the end workspace position

$$\mathbf{g}_{\text{end}} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

with 100 steps, by running `ex3/verify_linear_workspace_trajectory.py` the same way. Is the end effector workspace path a straight line? What about the joint angle plots? Does this make sense?

Note: You do not have to submit anything for this part; however, you should use this to detect any problems with your code before continuing!

e. **See it on the arm**

```
python local_autograder.py -p vis3
```

The arm moves between the same two poses twice, over and over: first along your `linear_joint_trajectory`, tracing its tip in **blue**, then along your `linear_workspace_trajectory`, tracing its tip in **gold**. The thin white line is the straight segment between the two end points. Only the gold trace should follow it; a straight line in joint space is a curve in the workspace. The status line shows how far the traced tip has strayed from the straight line so far, in millimetres; compare the two.

The `points` slider sets the number of points in each trajectory and `points / s` how fast the arm plays through them. Both take effect on the next pass. At higher

speeds the gold trace bows slightly off the line: that is the arm's servos lagging behind the commands, not your trajectory.

For the same reason as in `vis2`, the demo hands your `linear_workspace_trajectory` a copy of your `Robot` scaled up to a 3 m reach, with the goal scaled to match. Scaling every link of a planar arm scales every position and leaves every joint angle unchanged, so the angles it returns drive the real arm as they are.

Additional Notes: End Goal You may have noticed that the `linear_joint_trajectory` function takes a vector of joint angles, and the `linear_workspace_trajectory` function takes a workspace position.

Although this could be done in different ways, often the first function would be used when moving between joint "waypoints," rather than between a current robot position and a desired workspace position.

The workspace trajectory would be difficult to complete if it were between two joint configurations because one would have to enforce a continuous constraint on the joint angles in addition to the linear workspace constraint. This is not in general possible (as a concrete example, imagine an RR arm moving from an "elbow up" to an "elbow down" configuration where the end effector could not move, e.g., a straight line between two identical points in the workspace).

4) [15 points] Gravity Compensation on the xArm7

To hold an arm still, each joint's motor has to supply exactly the torque that gravity exerts about that joint. In this question you will compute those torques for two joints of the xArm7 using Jacobians, and check them against the simulated arm.

This part is done in simulation only. The real xArm7 has no joint torque sensors, and UFACTORY does not expose a way to command joint torques, so there is nothing on the real arm to check these numbers against. The simulator's `joint_torques` reports what a joint torque sensor would read.

a. **The model**

With joints 1, 3, 5 and 7 at zero, the elbow (joint 4) and the wrist (joint 6) of the xArm7 turn about axes parallel to the world y-axis, so everything they carry moves in the vertical x–z plane (x forward, z up). `ex4/grav_comp.py` models it as a planar RR arm:

- Link A is the forearm, from the elbow to the wrist, with length `LINK_A_LENGTH`. Its angle θ_a is measured from $+x$ toward $+z$.
- Link B is everything past the wrist (links 6 and 7 and the gripper). Its angle θ_b is measured relative to link A, also counter-clockwise.
- Each link's centre of mass is given in that link's own frame as a 2D offset, `COM_A` from the elbow and `COM_B` from the wrist: the first entry is along the link, the second perpendicular to it. The centres of mass are not exactly on the link lines, and ignoring the perpendicular part costs up to about 1 N·m.
- `MASS_A` and `MASS_B` are the masses each link carries.

All of these numbers are measured from the robot model the simulator runs, and are given at the top of the file. `planar_angles` (the arm's seven joint values to θ_a, θ_b) and `to_joint_torques` (planar torques to the arm's joint 4 and joint 6 torques) are provided; do not modify them.

With $R(\cdot)$ the 2×2 rotation matrix, the two centres of mass are at

$$\mathbf{p}_A = R(\theta_a)\,\mathbf{c}_A, \qquad \mathbf{p}_B = R(\theta_a)\begin{bmatrix} L_A \\ 0 \end{bmatrix} + R(\theta_a + \theta_b)\,\mathbf{c}_B$$

relative to the elbow, where L_A is `LINK_A_LENGTH` and $\mathbf{c}_A$, $\mathbf{c}_B$ are `COM_A`, `COM_B`.

b. [7 points] **Jacobians of the centres of mass**

Fill in `compute_jacobians(theta_a, theta_b)`. It returns `(J_A, J_B)`, the 2×2 Jacobians of $\mathbf{p}_A$ and $\mathbf{p}_B$ with respect to (θ_a, θ_b): rows are (x, z) and columns are (θ_a, θ_b). Note that θ_b does not move link A.

c. [8 points] **Gravitational torques**

Fill in `compute_gravitational_torques(theta_a, theta_b)`. A joint holding a mass m still against gravity has to push up on it with a force mg, so the torques the two joints must supply are

$$\boldsymbol{\tau} = (\mathbf{J}_A)^\top \begin{bmatrix} 0 \\ m_A g \end{bmatrix} + (\mathbf{J}_B)^\top \begin{bmatrix} 0 \\ m_B g \end{bmatrix}.$$

Return them as $[\tau_a, \tau_b]$.

d. **Verification**

`ex4/grav_comp.py` runs its own check when run as a script:

```
python ex4/grav_comp.py
```

It opens the simulator, levels the forearm, lowers the wrist 30 degrees at a time, and at each pose compares your torques on joints 4 and 6 with the simulated arm's `joint_torques`, printing a table like this:

Table 1: Gravity Compensation Verification Results

Joint 6 (deg)	Joint	Calculated (Nm)	Actual (Nm)	Diff. (Nm)	Result
30.00	4	XX.XX	XX.XX	XX.XX	PASS
30.00	6	XX.XX	XX.XX	XX.XX	PASS
60.00	4	XX.XX	XX.XX	XX.XX	PASS
60.00	6	XX.XX	XX.XX	XX.XX	PASS
90.00	4	XX.XX	XX.XX	XX.XX	PASS
90.00	6	XX.XX	XX.XX	XX.XX	PASS

A correct implementation passes every row, within 0.1 N·m.

Note: You do not have to submit anything for this part; however, you should use this to detect any problems with your code before continuing!

e. **See it on the arm**

```
python local_autograder.py -p vis4
```

This demo holds the arm in the same vertical plane. The `theta_a` and `theta_b` sliders pose the forearm and the wrist. Two **blue** balls mark where the model's constants put the two centres of mass, joined by a line through the wrist; they should sit inside the forearm and the wrist as the arm moves.

The status line puts your torque for each joint, evaluated at the angles the arm is actually at, beside the torque the simulated arm measures, and marks each PASS or FAIL. While the arm is still moving it says *settling* instead: the measured torque then includes the torque that accelerates the arm, which your gravity model doesn't account for. Watch how the torque on joint 4 changes as the forearm swings from level to vertical.

5 Submission Checklist

- ☐ Complete `ex1/robot.py`, `ex1/sample_velocities.py`, `ex1/jacobian_example.py`, `ex2/robot.py`, `ex3/robot.py`, `ex3/linear_joint_trajectory.py`, `ex3/linear_workspace_trajectory.py` and `ex4/grav_comp.py`.
- ☐ Confirm `python local_autograder.py` passes.
- ☐ Run `python create_submission.py` from the top level of the handout. It checks that your files are present and writes `handin.zip`; it does not grade anything.
- ☐ Upload `handin.zip` to the **HW4 Code** assignment on Gradescope.
- ☐ Create a PDF of your answers to the written questions named `writeup.pdf`, and upload it to the **HW4 Written** assignment on Gradescope.